fraction p of less significant weight indices from $\boldsymbol{\gamma}^{\tau}$ based on $\boldsymbol{\phi}_{\tau} = (\boldsymbol{\mu}^{\tau}, \boldsymbol{\sigma}^{\tau 2})$. In particular, we compute the Signal-to-Noise (SNR) of $|\boldsymbol{\theta}|$ to consider both the magnitude and variance of the weights during pruning, similar to (Kingma, Salimans, and Welling 2015). Additional details on SNR($|\boldsymbol{\theta}|$) are provided in the Appendix A.

After pruning step, we reintroduce the same fraction of weight indices into the sparse connectivity $\boldsymbol{\gamma}^{\tau+1}$ for the next ΔT steps. The method uses the highest absolute gradients of the weights, computed with a batch of inputs $\boldsymbol{x}$ of size B and stochastic weight sample $\boldsymbol{\theta}$. The grow criterion is given by:

$$\mathbb{E}_{q_{\boldsymbol{\phi}}} \left| \mathbb{E}_{\boldsymbol{x}} \frac{1}{B} \sum_{i=1}^{B} \nabla_{\boldsymbol{\theta}} \mathcal{L}(x_i) \right| .$$

We perform a one-step MC estimation to approximate the double expectation above, resulting in $|\sum_{i=1}^{B} \nabla_{\boldsymbol{\theta}} \mathcal{L}(x_i)|/B$.

We chose $\Delta T = 1000$ in our experiments to be consistent with (Liu et al. 2022; Li et al. 2024).

3.3 Sequential Ensembling Strategy

We propose a sequential ensembling procedure to obtain base learners (individual models part of an ensemble) $\{\boldsymbol{\theta}^1, \boldsymbol{\theta}^2, \cdots, \boldsymbol{\theta}^M\}$ that are collected in a single training run and used to construct the ensemble. The ensemble predictions are computed by averaging the predictions from each base learner. Specifically, if y_{new}^m denotes the outcome from the mth base learner, then the ensemble prediction for M base learners (for continuous outcomes) is given by $y_{\text{new}} = \frac{1}{M} \sum_{m=1}^{M} y_{\text{new}}^m$.

Our ensembling strategy generates a diverse set of base learners through a single end-to-end training process, which includes an exploration phase followed by M exploitation phases. During the exploration phase, we use a large constant learning rate for T_0 time steps to explore high-performing regions of the parameter space, aiming for sparse connectivity with strong predictive performance. At the conclusion of the exploration phase, the model sparsity and corresponding variational posterior parameters reach a good region on the posterior density surface.

Next, in each equally spaced exploitation phase ($\text{time} = t_{\text{ex}}$) of the ensemble training, we follow a two-step learning rate schedule: first applying a moderately large learning rate for $t_{\text{ex}}/2$ time followed by a small learning rate for the remaining $t_{\text{ex}}/2$ time. After the first model converges ($\text{time} = t_0 + t_{\text{ex}}$), which gives us our first sparse base learner, we prune a large fraction p_L of the sparse connectivity and then regrow same fraction of the weight indices according to our grow criterion. This approach helps the model escape current local minima and find a new performant subnetwork. We repeat this process $M-1$ times to generate M sparse base learners, including the first learner, which does not undergo the large prune-grow phase. Combining these individual sparse base learner results in our Sequential Bayesian Neural Subnetwork Ensemble (`SeBayS`). The pseudocode for `SeBayS` ensemble is provided in Algorithm 1.

We also perform ensembling over Bayesian subnetworks obtained in parallel, where each subnetwork undergoes a single exploration phase followed by a single exploitation phase. This results in a parallel Bayesian Neural Subnetwork Ensemble, which we refer to as the `BayS` ensemble. For baseline comparison, we also include a parallel ensemble of dense BNN models trained using the same strategy as the `BayS` ensemble, but with dense connectivity maintained throughout. We refer to this as the dense BNN ensemble.

Algorithm 1: **Se**quential **Bay**esian Neural **S**ubnetwork Ensembles (`SeBayS`)

Inputs: training data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$, ensemble size M, target sparsity s, set of initial sparse indices S_{init}, update interval ΔT, prune-grow rate p, large prune-grow rate p_L, exploration phase training time t_0, training time of each exploitation phase t_{ex}.
Model inputs: prior hyperparameter σ_0^2.
Output: Variational parameters $\boldsymbol{\phi} = (\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ estimates and sparse connectivities $\boldsymbol{\gamma}$.
Method: Set initial sparse connectivity γ_{init} using S_{init}; initialize variational parameters: $\boldsymbol{\phi}_{\text{init}} = (\boldsymbol{\mu}_{\text{init}}, \boldsymbol{\sigma}_{\text{init}}^2)$.
# Exploration Phase
for $t = 1, 2, \ldots, T_0$ **do**
  Update $\boldsymbol{\phi}^0 \leftarrow$ SGD($\mathcal{L}$).
  if (t mod ΔT) = 0 **then**
    Update $\boldsymbol{\gamma}^0$ using prune-grow rate p.
  end if
end for
# M Sequential Exploitation Phases
for $m = 1, 2, \ldots, M$ **do**
  for $t = 1, 2, \ldots, t_{\text{ex}}$ **do**
    Update $\boldsymbol{\phi}^m \leftarrow$ SGD($\mathcal{L}$).
    if (t mod ΔT) = 0 **then**
      Update $\boldsymbol{\gamma}^m$ using prune-grow rate p.
    end if
  end for
  Save the model with weights $\boldsymbol{\theta}^m$ sampled using $\boldsymbol{\phi}^m$.
  Update $\boldsymbol{\gamma}^m \rightarrow \boldsymbol{\gamma}^{m+1}$ using large prune-grow rate p_L.
end for

4 Experimental Results

In this section, we demonstrate the performance of our proposed `SeBayS` approach on network architectures and datasets commonly used in practice. Specifically, we use Wide ResNet28-10 model (Zagoruyko and Komodakis 2016) and train on the CIFAR-10 and CIFAR-100 datasets (Krizhevsky 2009). The models are trained with batch normalization, step-wise (piece-wise constant) learning rate schedules, and augmented training data. To ensure stable training, we applied KL annealing (Sønderby et al. 2016) and used a smaller learning rate for the variational variance parameter $\boldsymbol{\sigma}^2$ during exploration phase. We also conducted an extensive hyperparameter search, with the final settings provided in Appendix B. The details on fairness, uniformity, consistency in training and evaluation, and reproducibility considerations for `SeBayS` and other models are provided in Appendix B.

Baselines. Our baselines include a single deterministic deep neural network (DNN), variational inference trained sin-

Methods	Acc (↑)	NLL (↓)	ECE (↓)	cAcc (↑)	cNLL (↓)	cECE (↓)	# Training runs (↓)
Single Dense DNN Model*	96.0	0.159	0.023	76.1	1.050	0.153	1
Single Dense BNN Model	95.8	0.170	0.024	75.8	1.146	0.161	1
SeBayS (M = 1) (S = 0.8)	95.8	0.162	0.022	76.6	0.999	0.141	1
SeBayS (M = 1) (S = 0.9)	95.4	0.158	0.023	76.8	0.953	0.137	1
BayS (M = 1) (S = 0.8)	95.6	0.170	0.025	76.2	1.123	0.157	1
Monte Carlo Dropout*	95.9	0.160	0.024	68.8	1.270	0.166	1
MIMO (M = 3)*	**96.4**	0.123	0.010	76.6	0.927	0.112	1
EDST Ensemble (M = 3) (S = 0.8)*	96.3	0.127	0.012	77.9	0.814	0.093	1
EDST Ensemble (M = 7) (S = 0.9)*	96.1	0.122	0.008	77.2	0.803	0.081	1
SeBayS Ensemble (M = 3) (S = 0.8)	96.2	0.129	0.009	77.7	0.842	0.092	1
SeBayS Ensemble (M = 7) (S = 0.9)	96.2	**0.118**	**0.006**	**78.0**	**0.776**	**0.080**	1
TreeNet (M = 3)*	95.9	0.158	0.018	75.6	0.969	0.137	1.5
BatchEnsemble (M = 4)*	96.2	0.143	0.021	77.5	1.020	0.129	4
Rank-1 BNN (M = 4)†	96.3	0.128	**0.008**	76.7	0.840	0.080	4
LTR Ensemble (M = 3) (S = 0.8)*	96.2	0.133	0.015	76.7	0.950	0.118	4
Static Sparse Ensemble (M = 3) (S = 0.8)*	96.0	0.133	0.014	76.2	0.920	0.098	3
PF Ensemble (M = 3) (S = 0.8)*	96.4	0.129	0.011	78.2	0.801	0.082	6
DST Ensemble (M = 3) (S = 0.8)*	96.3	**0.122**	0.010	**78.8**	**0.766**	**0.075**	3
BayS Ensemble (M = 3) (S = 0.8)	**96.5**	0.127	0.010	78.2	0.852	0.095	3
Dense DNN Ensemble (M = 4)*	96.6	0.114	0.010	77.9	0.810	0.087	4
Dense BNN Ensemble (M = 3)	96.5	0.125	0.010	78.1	0.872	0.095	3

Table 1: Wide ResNet28-10/CIFAR-10: we mark the best results of one-pass efficient ensemble in bold and multi-pass efficient ensemble in blue. Results with * are obtained from (Liu et al. 2022). † Rank-1 BNN results are from (Dusenberry et al. 2020).

gle Bayesian neural network (BNN) (Blundell et al. 2015) (the top-performing BNN from the dense BNN ensemble), and the best individual Bayesian neural subnetworks from both the `BayS` and `SeBayS` ensembles. We also benchmark our method against several ensembling strategies, including Monte Carlo Dropout (Gal and Ghahramani 2016), rank-1 BNN Gaussian ensemble (Dusenberry et al. 2020), MIMO (Havasi et al. 2021), BatchEnsemble (Wen, Tran, and Ba 2020), TreeNet (Lee et al. 2015), static sparse ensemble of M static sparse networks, Lottery Ticket Rewinding (LTR) ensemble, pruning and fine-tuning ensemble (Han et al. 2015), DST and EDST ensembles (Liu et al. 2022), and ensembles of dense DNNs and BNNs. To ensure a fair comparison, all models were trained under consistent hardware, environment, data augmentation, and training schedules. We also imported many of the ensemble model results from (Liu et al. 2022). Further details on model implementation and learning parameters can be found in Appendix B.

Metrics. We quantify predictive performance and robustness focusing on the accuracy (Acc), negative log-likelihood (NLL), and expected calibration error (ECE) on the i.i.d. test data (CIFAR-10 and CIFAR-100) and corrupted test data (CIFAR-10-C and CIFAR-100-C) involving 19 types of corruption (e.g., added blur, compression artifacts, frost effects) (Hendrycks and Dietterich 2019). Additional details on the evaluation metrics are given in the Appendix B.

Results. The results for CIFAR-10 and CIFAR-100 experiments are presented in Tables 1 and 2, respectively. We choose the ensemble size M in our `SeBayS` approach same as the ones used in EDST method (Liu et al. 2022) for a fair comparison. We report the results for single training pass models in the upper half and multiple training pass models in the lower half of the Tables 1 and 2.

We included two `SeBayS` ensembles with different sparsity levels. In the first, we set $S = 0.8$, making 80% of the weights sparse and collecting $M = 3$ subnetworks for ensembling. In the second, we increased sparsity to $S = 0.9$, allowing us to collect $M = 7$ subnetworks within a similar training budget. In Table 1, we show that the `SeBayS` ensemble outperforms other single-pass models in terms of accuracy and robustness on CIFAR-10. Moreover, the `SeBayS` ensemble surpasses both the dense BNN ensemble and the `BayS` ensemble, achieving the lowest expected calibration error (ECE) among all the models for CIFAR-10.

In Table 2, we observe that the `SeBayS` ensemble with $S = 0.9$ achieves the lowest ECE on CIFAR-100 and CIFAR-100-C among the single pass models. Meanwhile, the `SeBayS` ensemble with $S = 0.8$ achieves the lowest negative log-likelihood (NLL) on CIFAR-100-C among the single pass models. Interestingly, the `BayS` ensemble achieves the lowest ECE on CIFAR-100 and CIFAR-100-C, as well as the lowest NLL on CIFAR-100-C among all the models, while maintaining strong predictive performance. Notably, both the `SeBayS` and `BayS` ensembles outperform the dense BNN ensemble on the ECE metric, with the `BayS` ensemble also yielding a lower NLL compared to the dense BNN ensemble.

Lastly, we have performed out-of-distribution (OoD) de-

Methods	Acc (↑)	NLL (↓)	ECE (↓)	cAcc (↑)	cNLL (↓)	cECE (↓)	# Training runs (↓)
Single Dense DNN Model*	79.8	0.875	0.086	51.4	2.700	0.239	1
Single Dense BNN Model	80.3	0.825	0.066	52.0	2.451	0.204	1
SeBayS (M = 1) (S = 0.8)	80.1	0.805	0.060	51.7	2.355	0.174	1
SeBayS (M = 1) (S = 0.9)	79.8	0.836	0.058	50.5	2.473	0.180	1
BayS (M = 1) (S = 0.8)	79.7	0.814	0.058	51.6	2.443	0.181	1
Monte Carlo Dropout*	79.6	0.830	0.050	42.6	2.900	0.202	1
MIMO (M = 3)*	82.0	0.690	0.022	53.7	2.284	0.129	1
EDST Ensemble (M = 3) (S = 0.8)*	82.2	0.672	0.034	**54.0**	2.156	0.137	1
EDST Ensemble (M = 7) (S = 0.9)*	**82.6**	**0.653**	0.036	52.7	2.410	0.170	1
SeBayS Ensemble (M = 3) (S = 0.8)	81.5	0.701	0.024	53.5	**2.148**	0.119	1
SeBayS Ensemble (M = 7) (S = 0.9)	81.8	0.676	**0.019**	53.1	2.205	**0.112**	1
TreeNet (M = 3)*	80.8	0.777	0.047	53.5	2.295	0.176	1.5
BatchEnsemble (M = 4)*	81.5	0.740	0.056	54.1	2.490	0.191	4
Rank-1 BNN (M = 4)†	81.3	0.692	0.018	53.8	2.240	0.117	4
LTR Ensemble (M = 3) (S = 0.8)*	82.2	0.703	0.045	53.2	2.345	0.180	4
Static Sparse Ensemble (M = 3) (S = 0.8)*	82.4	0.691	0.035	52.5	2.468	0.167	4
PF Ensemble*	83.2	0.639	0.020	54.2	2.182	0.115	3
DST Ensemble (M = 3) (S = 0.8)*	**83.3**	**0.623**	0.018	**55.0**	2.109	0.104	3
BayS Ensemble (M = 3) (S = 0.8)	82.0	0.664	**0.016**	54.8	**2.101**	**0.096**	3
Dense DNN Ensemble (M = 4)*	82.7	0.666	0.021	54.1	2.270	0.138	4
Dense BNN Ensemble (M = 3)	82.5	0.670	0.021	54.7	2.151	0.125	3

Table 2: Wide ResNet28-10/CIFAR-100: we mark the best results of one-pass efficient ensemble in bold and multi-pass efficient ensemble in blue. Results with * are obtained from (Liu et al. 2022). † Rank-1 BNN results are from (Dusenberry et al. 2020).

tection and adversarial robustness studies which we have summarized in Appendices C and D.

5 Ensemble Analysis

In this section, we demonstrate that the subnetworks converge to distinct local optima and exhibit functional behavior similar to that of independently trained sparse BNNs.

5.1 Function Space Analysis

Functional space analysis examines how neural networks map inputs to outputs within function space.

Quantitative Metrics. We measure the diversity of the sparse base learners in our `SeBayS` ensemble by quantifying the pairwise similarity of the base learner's predictions on the test data. The average pairwise similarity is given by $\mathcal{D}_d = \mathbb{E}\left[d(\mathcal{P}_1(y|x_1, \cdots, x_N), \mathcal{P}_2(y|x_1, \cdots, x_N))\right]$ where $d(.,.)$ is a distance metric between the predictive distributions and $\{(\boldsymbol{x}_i, y_i)\}_{i=1,\cdots,N}$ are the test data. We consider two distance metrics:

(1) *Disagreement:* the fraction of the predictions on the test data on which the base learners disagree: $d_{\text{dis}}(\mathcal{P}_1, \mathcal{P}_2) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(\arg\max_{\hat{y}_i} \mathcal{P}_1(\hat{y}_i) \neq \arg\max_{\hat{y}_i} \mathcal{P}_2(\hat{y}_i))$.

(2) *Kullback-Leibler (KL) divergence*: $d_{\text{KL}}(\mathcal{P}_1, \mathcal{P}_2) = \mathbb{E}\left[\log \mathcal{P}_1(y) - \log \mathcal{P}_2(y)\right]$.

When two models produce identical predictions for all the test data, then both disagreement and KL divergence are zero.

We report the results of the diversity analysis of the base learners that make up `SeBayS` and `BayS` ensembles ($M = 3, S = 0.8$) in Table 3 and compare them with the other sparse ensemble methods. We observe that `BayS` ensemble achieves diversity that surpasses all the methods except in CIFAR-10 case it is comparable to dense BNN ensemble and PF ensemble on KL metric. Our single pass `SeBayS` ensemble achieves diversity metrics which surpass the single pass EDST ensemble, multiple pass LTR, and Static ensembles, and are comparable with other single and multiple pass

	CIFAR-10 Experiment			CIFAR-100 Experiment		
Ensemble	d_{dis} (↑)	d_{KL} (↑)	Acc (↑)	d_{dis} (↑)	d_{KL} (↑)	Acc (↑)
LTR*	0.026	0.056	96.2	0.111	0.185	82.1
Static*	0.031	0.079	96.0	0.156	0.401	82.4
EDST*	0.031	0.073	96.3	0.126	0.237	82.2
MIMO*	0.032	0.086	96.4	–	–	82.0
PF*	0.035	**0.103**	96.4	0.148	0.345	83.2
DST*	0.035	0.095	96.3	0.166	0.411	**83.3**
SeBayS	0.035	0.082	96.2	0.148	0.280	81.5
BayS	**0.038**	0.102	**96.5**	**0.178**	**0.434**	82.0
DNN*	0.032	0.086	96.6	0.145	0.338	82.7
BNN	0.038	0.104	96.5	0.164	0.407	82.5

Table 3: Diversity metrics: prediction disagreement (d_{dis}) and KL divergence (d_{KL}) among sparse ensembles (M = 3, S = 0.8). Results with * are obtained from (Liu et al. 2022). DNN and BNN represent the dense DNN and BNN ensembles.